

NextFlow Assignment Interview Prep

Complete follow-up question bank with answers for the Galaxy.ai workflow builder assignment.

Core memory path: User signs in -> opens dashboard -> opens Trial Task Workflow -> runs graph -> backend executes DAG batches -> tasks call providers -> history stores node results -> UI displays output/errors.

Quick Things To Say First

- I built a Galaxy.ai-style LLM/image workflow builder using Next.js, React Flow, Clerk, Prisma/PostgreSQL, Trigger.dev, Gemini, and Groq.
- The most important technical part is DAG-based workflow execution with parallel batches.
- The required Crop Image node waits 30 seconds before returning.
- Run history stores node-level status, duration, output, and errors.
- Auth and data access are scoped to the actual signed-in Clerk user.

Area	What to remember
Frontend	Next.js App Router, React Flow canvas, Zustand state, Tailwind UI
Backend	Next.js API routes, Prisma, PostgreSQL, protected routes
Execution	Topological/DAG batching, Promise.all for parallel nodes, node-level error capture
Tasks	Trigger.dev tasks for crop, Gemini, Groq; local fallback for development
Auth	Clerk sign-in/sign-up, useUser profile, server auth scoping
Fixes	Correct API key config, actual user profile, clean sign-in URL, lint/build readiness

One-Minute Project Pitch

Use this answer when they ask: Tell me about your assignment.

Q: What did you build?

A: I built NextFlow, a Galaxy.ai-style workflow builder for LLM and image workflows. It has Clerk authentication, a dashboard for managing workflows, a React Flow canvas for connecting nodes, and run history for each execution.

Q: What is the main assignment requirement?

A: The assignment required three main pages: Clerk sign-in/sign-up, dashboard, and workflow canvas. It also required a sample workflow with Request Inputs, Crop Image nodes, Gemini LLM nodes, Response node, Trigger.dev execution, PostgreSQL persistence, run history, DAG execution, and a mandatory 30-second delay on Crop Image.

Q: Short pitch to memorize

A: NextFlow lets a signed-in user create, connect, run, and inspect AI workflow graphs. The backend executes the graph in dependency order, runs independent branches in parallel, calls image/LLM tasks, and stores node-level results in PostgreSQL.

Requirement Checklist

These are the points to mention if they ask whether the assignment is complete.

Q: Auth pages

A: Clerk sign-in and sign-up are implemented. Protected app routes redirect unauthenticated users to sign in.

Q: Dashboard

A: The dashboard lists workflows, shows last edited time/status, supports create/open/rename/delete/import, and includes the required sample workflow card.

Q: Canvas

A: The workflow canvas uses React Flow for draggable nodes, handles, edges, viewport controls, minimap behavior, and graph editing.

Q: Required nodes

A: Request Inputs, Crop Image, Gemini/Groq LLM, and Response nodes are implemented. Groq is included because the assignment allowed other providers.

Q: Sample workflow

A: The Trial Task Workflow is prebuilt with request inputs, two crop branches, multiple LLM steps, and a final response.

Q: Execution

A: Execution uses DAG/topological batching so independent nodes can run in parallel while respecting dependencies.

Q: History

A: Each workflow run is persisted with node-level status, duration, output, and errors.

Q: Persistence

A: PostgreSQL and Prisma store users, workflows, nodes, edges, runs, and node runs.

Q: Deployment

A: The app is deployed on Vercel and configured through environment variables.

Architecture

Explain the system from frontend to backend.

Q: What is the high-level architecture?

A: Next.js App Router handles pages and API routes. React components render the dashboard and workflow UI. Zustand stores client-side workflow state. React Flow renders the graph. API routes persist workflows and execute runs. Prisma talks to PostgreSQL. Clerk handles auth. Trigger.dev handles long-running/background tasks.

Q: Why Next.js?

A: It gives frontend routes, server-side API routes, middleware, deployment support on Vercel, and a clean App Router structure in one project.

Q: Why React Flow?

A: React Flow is built for node editors. It provides nodes, handles, edges, viewport, dragging, connection events, and minimap support so I did not have to build graph canvas primitives from scratch.

Q: Why Zustand?

A: Zustand is lightweight and works well for shared canvas state across toolbar, node panels, history panel, and the workflow builder without Redux boilerplate.

Q: Why Prisma and PostgreSQL?

A: The assignment required persistence. Prisma gives a typed schema and clean relational API, while PostgreSQL handles relational records plus JSONB fields for flexible node and edge data.

Q: Why Trigger.dev?

A: Crop Image has a required 30-second artificial delay and execution tasks can be long-running. Trigger.dev is designed for background/long-running jobs and gives task logs and retry behavior.

Important File Map

If they ask you to open code live, know where to go.

Q: `app/api/execute/route.ts`

A: Main workflow execution route. It validates user/workflow, creates a WorkflowRun, executes nodes in DAG batches, and stores NodeRun results.

Q: `server/execution/dag.ts`

A: Contains the topological batching logic used to decide execution order.

Q: `trigger/crop-image.ts`

A: Trigger.dev task for Crop Image. It waits 30 seconds, crops using Sharp, and returns output image data.

Q: `trigger/gemini.ts`

A: Trigger.dev task for Gemini execution and multimodal handling.

Q: `trigger/groq.ts`

A: Trigger.dev task for Groq text generation with fallback models.

Q: `prisma/schema.prisma`

A: Database models for User, Workflow, WorkflowNode, WorkflowEdge, WorkflowRun, and NodeRun.

Q: `store/workflow-store.ts`

A: Client workflow state: nodes, edges, viewport, add/delete/connect nodes, save workflow.

Q: `features/workflow`

A: Workflow builder UI, nodes, tabs, toolbar, panels, and history.

Q: features/dashboard

A: Dashboard and Galaxy shell/profile UI.

Q: middleware.ts

A: Route protection and redirect behavior for unauthenticated users.

Workflow Execution Deep Dive

This is the most important interview topic.

Q: What happens when the user clicks Run?

A: The frontend sends the workflow graph and selected scope to `/api/execute`. The backend checks the current user, verifies the workflow belongs to that user, creates a `WorkflowRun`, computes topological batches, executes each batch, stores node results, and returns the run summary to the UI.

Q: What is a DAG?

A: A DAG is a Directed Acyclic Graph. In this app, nodes are workflow steps and directed edges represent dependencies. Acyclic means there should be no circular dependency.

Q: What is topological sorting?

A: Topological sorting orders graph nodes so every node runs after its dependencies. In this app I use topological batching, which groups nodes that are ready at the same dependency level.

Q: How do you run nodes in parallel?

A: After computing a ready batch, nodes in that batch have no unresolved dependencies between them. I run them with `Promise.all`, so independent branches like the two crop nodes can execute at the same time.

Q: What happens if a node fails?

A: The node result is marked failed with its error message. Downstream nodes depending on it fail with an upstream dependency error. Independent branches can still complete if they do not depend on the failed node.

Q: How are node inputs resolved?

A: Each target input checks incoming edges by `targetHandle`. The source node output is read from the `nodeOutputs` map. Request Inputs are special because their field values are read directly from node data.

Q: Why store outputs in a map?

A: The execution route needs fast lookup of upstream results while executing downstream nodes.

Q: Why truncate stored output?

A: LLM output and image data can be large. Truncating keeps run history readable and prevents unnecessarily large database records.

Q: Can disconnected nodes run?

A: If included in the execution scope and they have no dependencies, they can appear in an early batch. In a production version I would add stronger validation around disconnected or invalid graph segments.

Q: How would you prevent cycles?

A: Use cycle detection during edge creation or before execution. If topological processing cannot consume all selected nodes, the graph has a cycle and execution should be rejected with a clear error.

Crop Image Node

Know this well because the assignment explicitly required the delay.

Q: Why does Crop Image wait 30 seconds?

A: The assignment specifically required Crop Image to add an artificial 30-second delay before returning.

Q: Where is the delay implemented?

A: It exists in the crop execution path and `Trigger.dev` crop task before the Sharp image processing work.

Q: What inputs does Crop Image accept?

A: It accepts an input image URL or data URL, plus x, y, width, and height values as percentages.

Q: How does cropping work?

A: The image is loaded into a buffer, Sharp reads the metadata, percentage crop values are converted into pixel dimensions, bounds are clamped, and Sharp extracts the selected region as PNG.

Q: Why Sharp?

A: Sharp is fast, widely used, and reliable for server-side image processing in Node.js.

Q: What does Crop Image return?

A: It returns a PNG as a base64 data URL so the next node can consume it as an image input.

Q: What happens if the input image is invalid?

A: The image fetch/load fails and the node is marked failed with an error in run history.

Q: How are crop bounds handled?

A: Coordinates and dimensions are clamped so the crop area stays inside the source image and has at least 1 pixel width/height.

Gemini and Groq

Answer confidently why both providers exist.

Q: Why Gemini?

A: Gemini is used for multimodal and vision-style workflow steps, especially when image inputs are connected.

Q: Why Groq?

A: Groq is used for fast text generation. The assignment allowed other AI providers, so Groq is acceptable for text-focused LLM use cases.

Q: Can Groq process images?

A: No, Groq text models do not process images in this implementation. Image/vision work should go through Gemini.

Q: How is Gemini called?

A: The code builds a request with text parts and optional inline image parts, then calls the Gemini generate content API or Trigger task.

Q: How is Groq called?

A: The code sends chat messages to Groq's OpenAI-compatible chat completions endpoint with system and user messages.

Q: How do you handle rate limits?

A: Groq tries fallback models when the response is 429. Gemini route can try fallback Gemini models or return a clear rate-limit error.

Q: Where are API keys stored?

A: API keys are stored in environment variables and must not be committed to GitHub.

Q: How would you secure keys in production?

A: Use Vercel environment variables, rotate exposed keys, restrict provider key permissions where possible, and never expose server API keys to client code.

Q: What caused the 401 issue in the review?

A: The screenshot showed Crop Image and LLM failing with 401 Invalid API key. Since Crop Image also failed, it was most likely Trigger.dev auth configuration, not only Gemini/Groq.

Q: What did you change after the 401 feedback?

A: I validated Trigger key type, ensured the correct Trigger project API key is used instead of a personal token, and verified env configuration locally before redeploying.

Trigger.dev

Be ready for questions on development mode and keys.

Q: What Trigger tasks did you create?

A: Crop Image, Gemini, and Groq tasks.

Q: What is Trigger mode?

A: In Trigger mode, the execution route triggers Trigger.dev tasks and polls their completion.

Q: What is local mode?

A: In local mode or fallback mode, the same work runs directly inside the Next.js API route for development/debugging.

Q: Why have local fallback?

A: It keeps development and demos from being completely blocked by external task auth problems, while still allowing Trigger.dev for real long-running tasks.

Q: Would you keep fallback in production?

A: I would keep it only if explicitly intended. In production, I would prefer strict Trigger mode so execution behavior is predictable and operationally observable.

Q: Personal token vs project key?

A: A personal token is for account-level CLI/API actions. A project runtime API key is what the app should use to trigger and run tasks for that project.

Q: How do you poll Trigger completion?

A: After triggering a task, the execution route polls the run handle until the task completes or fails, then stores the result/error.

Q: Is development mode okay?

A: For a trial assignment demo, Clerk/Trigger development mode is generally okay if the deployed flow works. For production, switch to production instances and keys.

Authentication and User Profile

This directly addresses HR's feedback.

Q: How is auth implemented?

A: Clerk handles sign-in/sign-up. Middleware protects app and API routes. Server routes use Clerk auth to identify the current user.

Q: Which routes are protected?

A: Dashboard, workflow pages, workflow APIs, run APIs, execute/import APIs, and provider/upload APIs are protected.

Q: How do you show the actual user?

A: The sidebar uses Clerk's useUser hook to display the signed-in user's full name, first name, email, and avatar image.

Q: What was the profile issue HR mentioned?

A: The recording made the profile look like a developer/demo profile. The fix is to display the actual logged-in Clerk user in the app UI and show it clearly in the demo.

Q: How is user data scoped on backend?

A: API routes get the current user ID and query/update workflows using both workflow ID and user ID. This prevents one user from reading or changing another user's workflows.

Q: What happens if Clerk keys are missing?

A: The app has a demo fallback for local development. In deployed assignment/demo mode, Clerk keys should be present so the real auth flow is used.

Q: Why did you shorten sign-in URL?

A: Protected route redirects were producing long encoded redirect URLs. I changed the middleware to redirect unauthenticated users to plain /sign-in for a cleaner demo flow.

Database and Prisma

Memorize the models and why they exist.

Q: What database models exist?

A: User, Workflow, WorkflowNode, WorkflowEdge, WorkflowRun, and NodeRun.

Q: Why separate WorkflowNode and WorkflowEdge?

A: Nodes and edges are separate graph entities. This makes it easy to update positions, data, and connections independently while preserving graph structure.

Q: What is stored in Workflow?

A: Workflow owner, name, description, viewport, status, timestamps, and relations to nodes, edges, and runs.

Q: What is stored in WorkflowNode?

A: Node ID, workflow ID, type, position JSON, node data JSON, and timestamps.

Q: What is stored in WorkflowEdge?

A: Edge ID, workflow ID, source, target, handles, edge data, and timestamp.

Q: What is WorkflowRun?

A: A single execution attempt for a workflow. It stores scope, state, inputs, start time, duration, and NodeRun children.

Q: What is NodeRun?

A: A per-node execution record containing node ID/title, state, duration, output, error, and start time.

Q: Why JSONB fields?

A: Node data, positions, viewport, edge metadata, and outputs vary by node type. JSONB keeps the schema flexible while PostgreSQL still stores it efficiently.

Q: What are indexes for?

A: Indexes on `userId` and `workflowId` speed up common queries like listing a user's workflows and fetching run history for a workflow.

Q: Why migrations?

A: Migrations make the database schema reproducible for reviewers, deployment, and future environments.

Frontend and State

Explain how the UI stays interactive.

Q: What is stored in Zustand?

A: Workflow graph data, nodes, edges, viewport, selected state, panels, and actions like add node, delete node, connect edges, and save workflow.

Q: Why not only React local state?

A: Many components need the same workflow state: toolbar, canvas, panels, node picker, and history. Zustand provides shared state without prop drilling.

Q: How do users add nodes?

A: The node picker calls the store's `addNode` action, which creates a typed node object with default inputs/outputs and inserts it near the current viewport.

Q: How do users connect nodes?

A: React Flow connection events create edges. The store validates source/target compatibility and saves the edge with `sourceHandle` and `targetHandle`.

Q: How is history shown?

A: The history panel fetches workflow runs and displays each run's node-level status, duration, output, and error details.

Q: How does import/export work?

A: A workflow can be serialized as JSON and imported back through an API route that creates corresponding workflow, node, and edge records.

Q: How did you match Galaxy.ai UI?

A: I mirrored the required layout: auth entry, dashboard shell/sidebar, workflow card layout, canvas, node UI, history panel, and visual styling from the reference.

API Routes

Short answers for route-by-route code review.

Q: `/api/execute`

A: Runs workflows. It performs auth, ownership check, run creation, DAG batch execution, task execution, error handling, and result persistence.

Q: `/api/workflows`

A: Lists workflows for the current user and creates new blank workflows.

Q: /api/workflows/[id]

A: Fetches, updates, or deletes a workflow after checking ownership.

Q: /api/workflows/seed

A: Creates or repairs the required Trial Task Workflow for the current user.

Q: /api/runs

A: Lists run history for workflows.

Q: /api/runs/[id]

A: Fetches a specific run with node-level details.

Q: /api/import

A: Imports workflow JSON and creates corresponding database records.

Q: /api/crop

A: Direct/local crop endpoint with auth and 30-second delay.

Q: /api/gemini

A: Direct Gemini API endpoint for LLM/vision generation.

Q: /api/groq

A: Direct Groq API endpoint for text generation.

Q: /api/transloadit/upload

A: Handles image upload through Transloadit with server-side signing.

Error Handling and Debugging

Use these for questions about failures.

Q: How are node errors handled?

A: Each node execution is wrapped in try/catch. Failures are captured as node-level errors and stored as NodeRun records.

Q: How did you debug the 401 issue?

A: I looked at which nodes failed. Since Crop Image and LLM both showed 401, the shared failure path was Trigger.dev auth. Then I checked environment key types and added validation around Trigger key configuration.

Q: How are missing env vars handled?

A: Routes check required environment variables and return clear errors. Trigger configuration also rejects a personal access token used in place of a runtime project key.

Q: How does the UI show failure?

A: Run history displays failed node status, duration, and error message so the reviewer can identify the exact failing step.

Q: What if one Promise in Promise.all fails?

A: Each node's async function catches its own error and returns a failed result object, so one rejected provider call does not crash the entire batch unexpectedly.

Q: What edge cases would you test?

A: Missing API keys, invalid image URL, graph cycle, disconnected node, failed upstream dependency, invalid workflow ID, unauthorized workflow access, and provider rate limit.

Security

They may push on API keys and auth.

Q: How do you protect secrets?

A: Secrets live in environment variables, not client code or Git. .env is ignored, and Vercel environment variables are used for deployment.

Q: What if keys were exposed?

A: Rotate them immediately in the provider dashboards and update Vercel/local env values.

Q: How do you prevent unauthorized access?

A: Every protected API route checks the current Clerk user and scopes queries by userId.

Q: What is SSRF risk here?

A: Fetching arbitrary image URLs from the server could be abused to reach internal resources. In production I would validate URL schemes, block private IP ranges, limit file size, and proxy uploads through controlled storage.

Q: How would you rate limit execution?

A: Add per-user rate limits, execution quotas, provider spend limits, and queue-level controls.

Q: How would you secure uploads?

A: Validate file type/size, scan or sanitize inputs, store in object storage, use signed URLs, and avoid trusting arbitrary remote URLs.

Performance and Production Improvements

Good answers for senior-style follow-up.

Q: Why run nodes in parallel?

A: It reduces total run time for independent branches. For example, two crop nodes with 30-second waits should complete in about 30 seconds together, not 60 seconds sequentially.

Q: Could API routes time out?

A: Yes, long-running synchronous work can hit platform limits. Trigger.dev is a better fit for long tasks and production execution.

Q: How would you improve live progress?

A: Use webhooks, server-sent events, polling, or realtime subscriptions to update node states as tasks complete instead of waiting for the full run response.

Q: How would you optimize image storage?

A: Do not store large base64 images in the database. Store images in object storage and save URLs/metadata in the DB.

Q: How would you scale for many users?

A: Move execution to background queues, use provider quotas, add indexes, store assets externally, cache static workflow data, and stream run updates.

Q: What would you improve next?

A: Production Clerk/Trigger keys, stronger cycle validation, better live progress, more tests, object storage for outputs, and observability around task execution.

Testing

Be honest but show you know what should be tested.

Q: What checks did you run?

A: I ran lint, TypeScript typecheck, production build, and Prisma schema validation.

Q: How would you unit test DAG execution?

A: Create small graphs with known dependencies and assert that topological batches match expected levels. Test parallel branches, single chain, cycle, and disconnected nodes.

Q: How would you test API routes?

A: Mock Clerk auth, Prisma, and providers. Verify unauthorized access, ownership checks, success paths, and error responses.

Q: How would you test provider calls?

A: Mock fetch responses for success, 401 invalid key, 429 rate limit, and malformed responses.

Q: How would you test UI?

A: Use Playwright to sign in, open dashboard, open sample workflow, run it, and confirm history output/status appears.

Q: What would you test for security?

A: Attempt accessing another user's workflow ID, missing auth, invalid imports, huge payloads, and malicious image URLs.

Live Interview Questions and Model Answers

These are likely quick-fire questions.

Q: Show me where execution happens.

A: Open `app/api/execute/route.ts`. The POST handler creates the run, computes batches, executes nodes, and persists results.

Q: Show me where the 30-second delay is.

A: Open `trigger/crop-image.ts` and the local crop path in `app/api/execute/route.ts`. The delay runs before Sharp crops the image.

Q: Show me where parallel execution happens.

A: In `app/api/execute/route.ts`, each topological batch is run with `Promise.all`.

Q: Show me where the signed-in profile is displayed.

A: Open `features/dashboard/galaxy-shell.tsx`. It uses `useUser()` to render name, email, and avatar.

Q: Show me where user auth is checked.

A: Open `middleware.ts` for route protection and `lib/current-user.ts` for backend user resolution.

Q: Show me the database schema.

A: Open `prisma/schema.prisma` and explain each model and relation.

Q: Show me where Groq is called.

A: Open `trigger/groq.ts` or `app/api/groq/route.ts`. The request goes to Groq's OpenAI-compatible chat completions endpoint.

Q: Show me where Gemini image input is handled.

A: Open `trigger/gemini.ts` or `app/api/execute/route.ts`. Image URLs are converted into inline Gemini image parts.

Q: Show me where history is saved.

A: In `app/api/execute/route.ts`, after execution, `WorkflowRun` is updated and `NodeRun` records are created.

Q: Show me import/export.

A: Open `app/api/import/route.ts` and the workflow UI/API tab where export/import actions are exposed.

Production-Ready Answer Bank

Use these when they ask what you would do differently.

Q: If this went to production tomorrow, what changes first?

A: Use production Clerk and Trigger environments, rotate keys, remove demo fallbacks, add strict cycle validation, object storage for images, better run progress updates, rate limiting, and monitoring.

Q: What observability would you add?

A: Structured logs per run/node, provider latency metrics, failure rate dashboards, Trigger task logs, and alerts for 401/429/provider errors.

Q: What would you change about data storage?

A: Store image outputs in S3/R2/Blob storage instead of base64 in DB. Keep DB records to URLs, metadata, status, and text outputs.

Q: How would you handle retries?

A: Retry transient provider failures with backoff, but avoid retrying validation/auth errors. Configure retry policy per node type.

Q: How would you handle cancellation?

A: Add run cancellation state, propagate cancellation to Trigger tasks if supported, and stop scheduling downstream batches.

Q: How would you handle collaboration?

A: Add organizations/team memberships, workflow sharing permissions, audit logs, and role-based access control.

Answers to HR Feedback

Use this if they ask about the earlier review comments.

Q: What was the execution issue?

A: The recording showed 401 Invalid API key. Because Crop Image also failed, I traced it to Trigger.dev key configuration rather than only Groq/Gemini.

Q: How did you fix execution?

A: I configured the correct Trigger project API key, added validation against using personal tokens in the runtime path, and verified the workflow execution flow again.

Q: What was the profile issue?

A: The profile in the demo should represent the actual signed-in app user, not a developer/deployment identity.

Q: How did you fix profile?

A: The app sidebar and account menu now read Clerk's signed-in user through `useUser()` and display real name/email/avatar.

Q: Why are Clerk/Trigger in development mode?

A: This is a trial assignment demo, so development/test mode is acceptable for demonstration. For production, I would switch both to production instances and keys.

Final 30-Second Closing Answer

Yes, the assignment requirements are covered. The app has Clerk auth, dashboard, workflow canvas, required node types, a prebuilt Trial Task Workflow, DAG-based execution with parallel branches, a 30-second Crop Image delay, Trigger.dev task integration, Prisma/PostgreSQL persistence, and run history with node-level results. Groq is included for allowed fast text generation, while Gemini handles multimodal/vision-style steps. For production I would switch Clerk and Trigger to production mode, add stronger graph validation, object storage for image outputs, live run updates, rate limiting, and more automated tests.